MDPI

Article

Short-Response Collapse in Countdown RLVR: Training Instability, Format Effects, and Limited Transfer

Yongsheng Lian [1]

[1] Department of Mechanical Engineering, University of Louisville, Louisville, KY 40292, USA; yongsheng.lian@louisville.edu

Abstract

Reinforcement learning with verifiable rewards can raise in-domain scores, but its stability and transfer remain uncertain. We study Qwen2.5-1.5B-Instruct fine-tuning on Countdown, centered on a 22-run DAPO-style collapse screen; selected PPO- and GRPO-style checkpoints serve as observational references. Sixteen runs produced responses averaging fewer than 100 tokens by step 100. All five dynamic-sampling runs using equal data and rollout seeds 2–6 collapsed at step 20. A targeted current-environment replay of the legacy configuration with its default seed pair remained healthy, averaging 1,527 tokens over steps 90–100 and reaching 47.2% validation accuracy. Matched no-dynamic-sampling runs also varied across seeds. Selected healthy checkpoints raised Countdown validation accuracy from 3.0% for the base model to 66.0–75.1%, but collapsed and healthy members of matched recipes diverged at long horizons. The largest selected-checkpoint gain in format-tolerant GSM8K accuracy was approximately three points, with no consistent improvement on MATH, BBH, or MMLU-Pro. A mandatory final-answer prompt increased the unchanged base model's strict GSM8K score by 25.2 points while reducing tolerant accuracy by 5.2 points. The evidence establishes run-dependent training instability and strong format effects, but not its precise stochastic cause, broad reasoning transfer, or an algorithm ranking.

Keywords: reinforcement learning with verifiable rewards; large language models; GRPO; DAPO; training collapse; reasoning transfer

1. Introduction

Reinforcement learning with verifiable rewards (RLVR) has become a prominent approach to eliciting mathematical and symbolic reasoning from large language models (LLMs). Systems such as OpenAI o1 [1] and DeepSeek-R1 [2] demonstrate that post-training can substantially change generated reasoning behavior. However, higher task reward does not by itself establish that a model has acquired broadly transferable reasoning capability. An apparent gain can instead arise from better compliance with an answer format, a redistribution of probability toward solutions already available in the base model, or selection of an unusually successful training seed. Community reproductions such as TinyZero made Countdown a prominent small-model testbed for R1-Zero-style training [22]; this motivates auditing whether its apparent reasoning gains are stable and transferable.

A variety of policy-optimization recipes have been proposed for LLM post-training. Proximal Policy Optimization (PPO) uses a learned critic and clipped policy updates [3].

Received:
Accepted:
Published:

Copyright: © 2026 by the author. Submitted to *AI* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Group Relative Policy Optimization (GRPO) replaces the critic with advantages normalized over responses to the same prompt [4]. Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO) modifies clipping and loss aggregation and uses dynamic sampling to retain prompts whose response groups contain reward variation [5]. These methods are usually reported with different models, datasets, reward functions, rollout budgets, and optimization schedules, so comparisons between their published headline scores do not isolate algorithmic effects.

This single-model, single-training-domain case study centers on DAPO-style stability experiments using Qwen2.5-1.5B-Instruct and Countdown; selected PPO- and GRPO-style checkpoints provide contextual, not algorithm-controlled, references. We investigate four questions: (1) how frequently early short-response collapse occurs across runs and environments; (2) whether dynamic sampling prevents collapse when training exposure is controlled; (3) how reward composition, normalization, clipping, and KL regularization relate to the survival of reasoning behavior; and (4) whether downstream gains reflect problem-solving transfer, answer-format compliance, or increased probability assigned to solutions already present in the base model's sampling distribution.

Our results reveal substantial run-dependent instability. A screen of 22 runs finds early short-response collapse in 16 runs, including runs with and without dynamic sampling. All five screened dynamic-sampling runs in the current environment, which used equal data and rollout seed pairs 2–6, collapsed; however, a targeted replay of the reconstructed legacy configuration in the same current environment with the legacy default seed pair (1, 0) remained healthy through step 100. The replay rules out a deterministic current-environment explanation but does not isolate which random stream or residual reconstruction difference caused the contrast. In the legacy headline comparison, disabling dynamic sampling also increased the number of trained prompts and optimizer updates by a factor of six, precluding a causal attribution to sampling alone. Under format-tolerant evaluation, the strongest individual checkpoints improve GSM8K modestly but do not consistently improve MATH, BBH, or MMLU-Pro. A prompt-only format control produces a large strict-score increase while reducing tolerant accuracy, confirming that presentation alone can move the strict benchmark substantially. These results motivate evaluating RL recipes through training stability, behavioral diagnostics, compute exposure, and held-out transfer rather than training reward alone.

Relative to the preliminary preprint [13], this article adds the multi-run collapse screen, a targeted frozen-environment replay, explicit environment stratification, format-tolerant rescoring, a prompt-only format control, pass@k analysis, scorer-precedence corrections, and artifact manifests. It also narrows the earlier algorithm-comparison framing: PPO and GRPO are now reference recipes, and claims about dynamic sampling, training stability, and transferable reasoning are limited to what the matched evidence supports.

The remainder of this paper is structured as follows. Section 2 situates the study in recent RLVR work. Section 3 defines the optimization recipes, and Section 4 describes training and evaluation. Section 5 reports training diagnostics, ablations, collapse rates, transfer, and capability coverage. Sections 6 and 7 discuss the scope and implications of the findings.

2. Related Work

Critic-free optimization and loss normalization. GRPO removes the learned critic used by PPO and estimates advantages from response groups [4]. Subsequent work has shown that implementation details are consequential: REINFORCE++ uses global advantage normalization [6], while the Dr. GRPO analysis identifies response-length bias induced by common GRPO normalization choices and proposes an alternative objective [7]. DAPO

combines asymmetric clipping, dynamic sampling, token-level loss aggregation, and overlong-response handling [5]. These studies motivate our decision to treat named methods as complete recipes unless sampling budget, aggregation, reward, and update exposure are matched.

Reinforcement-learning reproducibility studies show that apparent improvements can depend strongly on random seeds, implementation details, and the statistical unit of analysis [26–28]. This literature motivates our separation of benchmark-item uncertainty from training-run uncertainty and our refusal to infer an algorithm ranking from single selected checkpoints.

Sampling efficiency and collapse. The open-source TinyZero project used Countdown and multiplication as small-scale R1-Zero reproduction tasks [22], helping establish the setting audited here. Dynamic sampling excludes homogeneous reward groups so that retained prompts produce a learning signal, but additional rollouts can be expensive. Selective-rollout methods instead attempt to predict and skip uninformative prompts before generation [8]. A complementary line of work analyzes entropy collapse and proposes targeted clipping or KL control for high-covariance tokens [9]. Our study contributes a seed-level screen of a different observable failure mode—collapse to extremely short responses—and tests whether dynamic sampling, reward changes, overlong filtering, or KL regularization reliably prevent it.

Reward validity, elicitation, and transfer. Recent evidence cautions against equating benchmark improvement with newly acquired reasoning. Spurious or format-based rewards can elicit large gains in some Qwen models, with weaker effects in other model families [10]. Large-k sampling studies similarly report that RLVR often improves pass@1 while failing to expand, and sometimes narrowing, the solution coverage of the base model [11]. Broader evaluations of math-tuned models find that transfer beyond the training domain is uneven and depends on the post-training method [12]. We connect these concerns by jointly reporting format-tolerant scores, strict-format decomposition, held-out transfer, and pass@k after Countdown training.

3. Background: Optimization Recipes

This section reviews the formulations of PPO, GRPO, and DAPO needed to distinguish the implemented recipes; the empirical contribution begins in Section 5. Extended derivations are available in Ref. [13].

PPO [3] is widely used for RLHF, but its learned value function adds memory and optimization overhead. GRPO [4] and DAPO [5] remove the critic in favor of group-relative advantage estimation. We summarize the three objectives to identify the recipe differences examined empirically.

3.1. Proximal Policy Optimization

PPO replaces TRPO's constrained second-order update [24] with a first-order clipped surrogate objective [3].

$$\mathcal{J}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t\hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \tag{1}$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$ is the likelihood ratio evaluated at the sampled state–action pair (s_t, a_t), $\hat{A}_t = \hat{A}^{\pi_{\theta_{\mathrm{old}}}}(s_t, a_t)$ is the estimated advantage (GAE for the displayed PPO recipe [25]), $\pi_{\theta_{\mathrm{old}}}$ is the pre-update policy, and π_θ is the updated policy. The first argument of min is the ordinary importance-weighted policy-gradient contribution; $\mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)$ truncates the ratio to the displayed interval, where ϵ is the symmetric clipping width.

Clipping and regularization. Clipping limits extreme likelihood-ratio contributions to the surrogate objective but does not impose a hard trust region [14,15]. In our implementation, the PPO actor minimizes

$$L(\theta, \phi) = -\mathcal{J}^{\text{CLIP}}(\theta) + c_1 L^{\text{critic}}(\phi) - c_2 \mathcal{H}(\pi_\theta), \tag{2}$$

where L^{critic} trains the value model and c_2 controls the optional entropy bonus. The broader PPO sweep also varies a per-token KL reward penalty against the reference model; the headline PPO checkpoint in Tables 1 and 2 has that penalty disabled. Thus, the reported settings are complete recipes rather than estimates of one abstract PPO algorithm.

3.2. Group Relative Policy Optimization

For each prompt, GRPO samples G responses and replaces the critic with within-group relative advantages [4]:

$$\widehat{A}_i = \frac{R_i - \overline{R}}{\text{std}(R_1, \ldots, R_G) + \epsilon_A}, \qquad \overline{R} = \frac{1}{G} \sum_{j=1}^{G} R_j. \tag{3}$$

A conventional GRPO presentation averages the clipped token objective within each response and then across the G responses. That response-normalized form is background and is not the aggregation used by the headline GRPO checkpoint. All headline group-relative runs use Verl's global `token-mean` aggregation. Defining the clipped token contribution as $\ell_{i,t}$, their implemented actor objective has the form

$$\mathcal{J}_{\text{impl}}(\theta) = \mathbb{E}\left[\frac{1}{\sum_{i,t} m_{i,t}} \sum_{i,t} m_{i,t} (\ell_{i,t} - \beta k_{i,t}) \right], \tag{4}$$

where $\ell_{i,t} = \min\Big(r_{i,t} \widehat{A}_i, \text{clip}(r_{i,t}, 1-\epsilon, 1+\epsilon) \widehat{A}_i \Big)$, $m_{i,t}$ masks padding and otherwise selects generated tokens, and $k_{i,t}$ is Verl's per-token low-variance KL estimator when KL loss is enabled. For the headline DAPO recipes $\beta = 0$; for the strong-KL GRPO checkpoint $\beta = 0.04$. The A1 ablation additionally masks truncated responses, but A1 is not a headline checkpoint in Tables 1–2.

If every reward in a group is identical, Equation 3 is zero. A small nonzero reward difference can nevertheless become an order-one standardized advantage. For example, with $G = 8$, seven rewards of 0 and one format-only reward of 0.1 give $\overline{R} = 0.0125$. Verl's implementation uses PyTorch's default Bessel-corrected sample standard deviation, giving $s \approx 0.0354$ and a format-only advantage of $(0.1 - 0.0125)/(0.0354 + \epsilon_A) \approx 2.47$ when ϵ_A is negligible. Thus, group standardization can make a minor format credit a strong positive update signal. This calculation identifies a plausible pressure toward reward-bearing shortcuts, not a demonstrated cause of collapse. In particular, global token-mean aggregation weights generated tokens equally and does not give a short response greater weight per token; establishing the temporal mechanism requires token-level entropy and format-validity diagnostics.

3.3. Decoupled Clip and Dynamic Sampling Policy Optimization

The DAPO-style recipe uses the implemented token-mean objective in Equation 4 with asymmetric ratio clipping, dynamic sampling, and optional overlong-response handling [5]. Here the symmetric interval inside $\ell_{i,t}$ is replaced by $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$; the recorded widths are reported in Table 1. Dynamic sampling retains prompt groups containing reward variation and resamples when too few informative groups remain. It neither balances correct and incorrect responses nor deletes correct responses from a mixed

group. Global token aggregation changes length weighting; it does not create token-level rewards because all tokens share the response-level outcome advantage.

3.4. Recipe Comparison

PPO uses generalized advantage estimation and a learned critic; GRPO and DAPO use group-relative outcome advantages. The implementations also differ in clipping, KL placement, aggregation, group size, response cap, reward, and update exposure. We reserve causal language for explicitly matched ablations and otherwise refer to PPO-, GRPO-, and DAPO-style recipes.

4. Materials and Methods

We fine-tune Qwen2.5-1.5B-Instruct [16] on Countdown and evaluate on GSM8K [17], MATH [18], BBH [19], and MMLU-Pro [20]. The complete test sets contain 1,319, 5,000, 6,511, and 12,032 items, respectively.

4.1. Evaluation Protocol

The *standard* condition uses a benchmark-specific direct prompt and is the primary transfer measure. The *think* condition requests Countdown-style reasoning and answer tags and probes whether the trained format survives out of domain. Main-table generations use vLLM greedy decoding (temperature 0), no repetition penalty, and at most 8,192 output tokens.

The strict scorer requires the answer in the last explicit final-answer marker. Format-tolerant correctness is a strict superset: a correct explicit marker is always accepted, and a marker-free fallback is applied only when generation completed without length truncation. The fallback uses the last number for GSM8K, symbolic extraction over the response for MATH, or the last choice for BBH and MMLU-Pro. Tolerant accuracy is primary; strict accuracy combines correctness and format compliance. BBH and MMLU-Pro use this custom zero-shot protocol rather than their conventional few-shot protocols.

For pass@k, we sample at temperature 0.6 and top-$p = 0.95$ with a 4,096-token limit, drawing 64 responses per MATH-500 item and 32 per GSM8K item. We report the unbiased estimator. Paired tests compare fixed checkpoints and do not estimate training-seed uncertainty. Appendix A gives the exact prompt strings, chat wrapping, dataset identifiers, stopping behavior, and scorer precedence. Although immutable revisions were not recorded initially, exact canonical-row comparison recovered the MATH, BBH, and MMLU-Pro revisions; their loaders are now pinned. The upstream GSM8K revision remains unknown, but the exact evaluated parquet is vendored and hashed. A machine-readable manifest and offline verifier accompany the artifact. A deterministic stratified code-and-record audit exposed two scorer-precedence errors: later incidental text could override a correct GSM8K marker, and unfinished length-truncated work could receive fallback credit. We corrected both rules, rescored the archived outputs, and added regression checks; among the five headline checkpoints, any individual benchmark accuracy changed by at most 0.91 percentage points. A blinded 40-record audit packet now targets the remaining GSM8K concern: 24 base-model responses scored correct only by the last-number fallback, eight base marker-free incorrect controls, four base strict-correct controls, and four trained-model fallback-only cases. Independent annotation has not yet been completed, so we do not claim a human agreement or scorer-error rate. In particular, the approximately three-point tolerant GSM8K gain remains conditional on the validity of the last-number fallback.

4.2. Training Setup and Headline Configurations

All runs initialize from Qwen2.5-1.5B-Instruct and use Countdown as the sole training task [21]. The DAPO-style scalar reward is the sum of a 0.1 credit for the full reasoning-and-answer tag structure and a 0.9 correctness credit. Thus a correct but format-invalid response scores 0.9, whereas a fully formatted incorrect response scores 0.1. The selected PPO and GRPO runs use an extraction-based reward: any correct extracted `<answer>` expression scores 1.0 regardless of the full reasoning-tag structure, any extracted but wrong or invalid expression scores 0.1, and a response with no extracted expression scores 0. These rewards share the numerical values 0, 0.1, and 1.0 but differ in what earns the 0.1 component and whether correct format-invalid responses score 0.9 or 1.0. Training uses Verl [23], one actor epoch per batch, AdamW, and rollout temperature 1.0.

Implementation details and hardware. The recorded current-environment ablations used one node with four NVIDIA RTX A6000 GPUs (48 GB each), bfloat16 rollout weights, PyTorch 2.6.0+cu124, and vLLM 0.8.5.post1. FSDP optimization used AdamW with the framework defaults $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The current-environment DAPO-style actor used weight decay 0.1, a 10-step linear warmup to the reported learning rate, and a constant schedule thereafter. The archived headline PPO and GRPO actor configurations used weight decay 0.01, zero warmup, and a constant schedule; the PPO critic likewise used weight decay 0.01 and zero warmup. The archived legacy configurations do not preserve every environment field, so current-environment specifications are not retroactively assigned to them. In the DS configuration, a group was rejected only when all eight response-level accuracy values were identical. The retry limit was non-positive (no fixed numerical cap): sampling continued until 32 informative prompt groups were collected or the finite data iterator was exhausted; the archived run required at most three generation rounds in an observed step.

A dash in Tables 1 and 2 means not applicable; NR means unavailable. The legacy DS configuration omitted explicit seed overrides; the archived framework defaults were data seed 1 and rollout seed 0, although the exact runtime random-number-generator state was not preserved. The 22-run screen contains current-environment seed labels 1–6 for A1 and matched no-DS, current-environment equal data/rollout seed pairs 2–6 plus one legacy run for DS, and current-environment seeds 3–6 for A1+KL. The legacy DS run is displayed as run 1 only for table ordering. A targeted post-screen replay used the reconstructed legacy configuration, including the default seed pair, in the current environment; it is reported separately and is not added to the 22-run denominator. The current environment uses Verl-fork with PyTorch 2.6.0 and vLLM 0.8.5.post1. A side-by-side legacy/current record in `environment/DS_LEGACY_CURRENT_COMPARISON.md` distinguishes matched settings from unverified legacy fields.

Table 1. Recorded optimization settings of the headline checkpoints. Clip ".20/.28" denotes $\epsilon_{\text{low}} = 0.20$ and $\epsilon_{\text{high}} = 0.28$; symmetric ".20/.20" uses 0.20 on both sides.

Recipe (step)	Estimator	Actor LR	Critic LR	KL	Ent.	Clip	Aggregation	G	Reward	Cap
DAPO DS (300)	GRPO	10^{-6}	–	0	0	.20/.28	token mean	8	format+corr.	4096
DAPO original no-DS (300)	GRPO	10^{-6}	–	0	0	.20/.28	token mean	8	format+corr.	4096
DAPO matched no-DS (300)	GRPO	10^{-6}	–	0	0	.20/.28	token mean	8	format+corr.	4096
GRPO strong-KL (250)	GRPO	10^{-6}	–	.04	0	.20/.20	token mean	2	extract+corr.	2048
PPO (520)	GAE	10^{-6}	10^{-5}	0	0	.20/.20	token mean	1	extract+corr.	2048

https://doi.org/10.3390/ai1010000

Table 2. Recorded per-step training exposure. Updates are inferred for one actor epoch; the DS generated-prompt count is the observed mean. Generated and retained response counts are separated because dynamic sampling makes them unequal.

Recipe	Gen. prompts	Gen. responses	Trained prompts	Trained responses	Mini-batch	Updates/step	Step	Wall time
DAPO DS	200.3 mean	∼1602	32	256	32	1	300	74.9 h
DAPO original no-DS	192	1536	192	1536	32	6	300	70.4 h
DAPO matched no-DS	32	256	32	256	32	1	300	NR
GRPO strong-KL	192	384	192	384	64	3	250	NR
PPO	192	192	192	192	16	12	520	NR

These data were reconstructed from configurations and logs, not a preregistered protocol. Generated- and trained-token counts and GPU-hours are unavailable. Wall time is NR when the legacy job's complete start/end timestamps were not retained; it is not an estimate of zero or equal compute. Cross-family comparisons remain observational, and the original DS/no-DS contrast cannot identify the effect of filtering.

5. Results

We first summarize component diagnostics and then present the collapse screen, transfer, format decomposition, and pass@k. Unless stated otherwise, downstream accuracies use the standard prompt and format-tolerant scoring. Most full-length configurations have one seed and do not support population-level method rankings.

5.1. Exploratory Recipe Diagnostics

PPO learning rate and regularization. The archived PPO grid is incomplete and cannot isolate entropy or KL effects. Its directly observed downstream values are therefore reported without a component-level causal claim in Appendix Table A3. Every evaluated PPO run at 3×10^{-6} performed substantially below the base model on the four-benchmark standard-prompt mean, whereas the selected 10^{-6} run retained more performance. These single-seed configurations identify high-risk recipes rather than universal effects.

GRPO group size and KL. The $G = 2$, 4, and 8 runs generated 384, 768, and 1,536 responses per step, so group size is confounded with rollout budget and no independent group-size effect is claimed. The single-seed KL sweep is reported in Appendix Table A3; it shows a sharp prompt-conditioned contrast but does not calibrate a general threshold.

Aggregation, DAPO group size, and dynamic sampling. Response-length differences cannot be attributed to aggregation because KL, reward, response cap, and rollout budget also change. DAPO group-size comparisons likewise change response budget, and $G = 2$ also changes KL. The original DS/no-DS runs change both filtering and exposure. In matched-exposure runs, collapsed and non-collapsed seeds occur with and without DS; the recorded DS run also required up to three generation rounds for some steps.

5.2. Short-Answer Shortcut under the Think Prompt

Weak-KL GRPO runs learned a prompt-conditional shortcut: under the Countdown-style think prompt they often emitted only a final-answer tag, with median responses near 20 characters, while standard-prompt performance remained much closer to the base model. Stronger KL runs retained longer responses. This behavior is evidence that the reward can be optimized without preserving the requested reasoning trace, but the present sweep does not identify a single cause: KL, loss aggregation, reward definition, and sampling configuration differ across recipe families. The format credit also did not prevent early collapse in the multi-seed screen. We therefore report response length and prompt-conditioned behavior as diagnostics rather than treating verbosity as a proxy for reasoning quality.

5.3. Early Short-Response Collapse across Seeds

We use *early short-response collapse* as an operational screening label, not as a direct measure of reasoning quality. The primary definition is a mean generated response length below 100 tokens over steps 90–100. Sixteen of 22 runs met this criterion. The label was often associated with answer-only behavior, but not invariably with lower Countdown validation accuracy: some endpoint-healthy no-DS seeds had validation accuracy comparable to collapsed seeds, and the longest runs also had substantial length truncation. Concise correct reasoning and long incoherent reasoning remain possible; response length alone is therefore insufficient to establish capability loss.

Table 3. Early short-response screen at the 100-token threshold. Wilson intervals describe binomial uncertainty within each recipe. A1 denotes correctness-only reward with overlong-response filtering. The targeted DS replay was conducted after this screen and is excluded from its counts and inferential summaries.

Recipe	Collapsed	Total	Rate (%)	Wilson 95% CI (%)
Dynamic sampling (DS)	5	6	83.3	43.6–97.0
Matched no-DS	3	6	50.0	18.8–81.2
A1	5	6	83.3	43.6–97.0
A1 + KL 0.01	3	4	75.0	30.1–95.4
All screened runs	16	22	72.7	51.8–86.8

A threshold-sensitivity check yielded 14/22, 16/22, and 17/22 labeled runs at 50, 100, and 200 tokens, respectively. The no-DS count changed from 1/6 at 50 tokens to 3/6 at 100 and 200 tokens; DS and A1 remained 5/6 at all three thresholds, while A1+KL changed from 3/4 to 4/4 only at 200 tokens. Thus, the qualitative observation of frequent very short outputs is robust overall, but the recipe-specific no-DS rate is threshold-sensitive. In the original screen, the one healthy DS run was also the only legacy-environment DS run; all five current-environment DS runs, which used equal data and rollout seed pairs 2–6, collapsed at the same step-20 event window. A targeted current-environment replay of the reconstructed legacy configuration with its default seed pair (1, 0) subsequently remained healthy through step 100 (Table 4). This result rules out the hypothesis that the current environment deterministically produces DS collapse. Because only one targeted replay was performed and both seed streams differ from the failed runs, it neither identifies the responsible stochastic factor nor estimates a DS failure probability. The current-environment matched no-DS recipe also varied sharply by seed. In the original 22-run screen, the recipe-specific Wilson intervals overlap substantially, and a two-sided Fisher exact comparison of DS and matched no-DS gives $p = 0.55$. With only four to six runs per arm, this test is severely underpowered; these data establish instability, not a statistically resolved ordering among recipes.

The default seed pair did not guarantee survival in the current environment: A1 run 1 remained healthy whereas matched no-DS run 1 collapsed, so the cross-recipe evidence argues against a generic explicit-seed-handling artifact while leaving the DS-specific mechanism unresolved.

We also performed a descriptive post-hoc time-to-event analysis on the archived per-step histories. Five-step trailing windows were evaluated at steps 10, 15, ..., 100. The event time is the first window with mean response length below 100 tokens when the next two windows are also below 100; runs without an event are right-censored at step 100. As shown by the multi-panel trajectories in Figure 1, 17 of 22 runs had such a sustained short-response episode. The five current-environment DS events occurred by step 20; A1 events occurred at steps 25 (four seeds) and 50 (one seed); matched no-DS events occurred

Table 4. Targeted DS replay compared with the archived DS evidence. The replay was conducted after the 22-run screen and is excluded from Table 3, its Wilson intervals, and the Fisher test. Endpoint length is the mean over steps 90–100; accuracy is sampled Countdown validation accuracy at step 100. Ranges summarize the five current-environment screen runs.

Run group	Environment	Data seed	Rollout seed	Endpoint length	Accuracy (%)	Status
Archived healthy DS	Legacy	Default 1	Default 0	1569.4	50.8	Healthy
DS screen runs ($n = 5$)	Current	2–6	2–6	19.5–20.6	26.2–28.8	Collapsed
Targeted legacy-config replay	Current	1	0	1527.4	47.2	Healthy

at steps 30, 45, and 50; and A1+KL events occurred at steps 25 or 30. One A1+KL seed subsequently recovered to a 132-token mean over steps 90–100 and was endpoint-healthy. Thus, the event denotes the first sustained episode, not an irreversible absorbing state, and it answers a different question from the primary endpoint label.

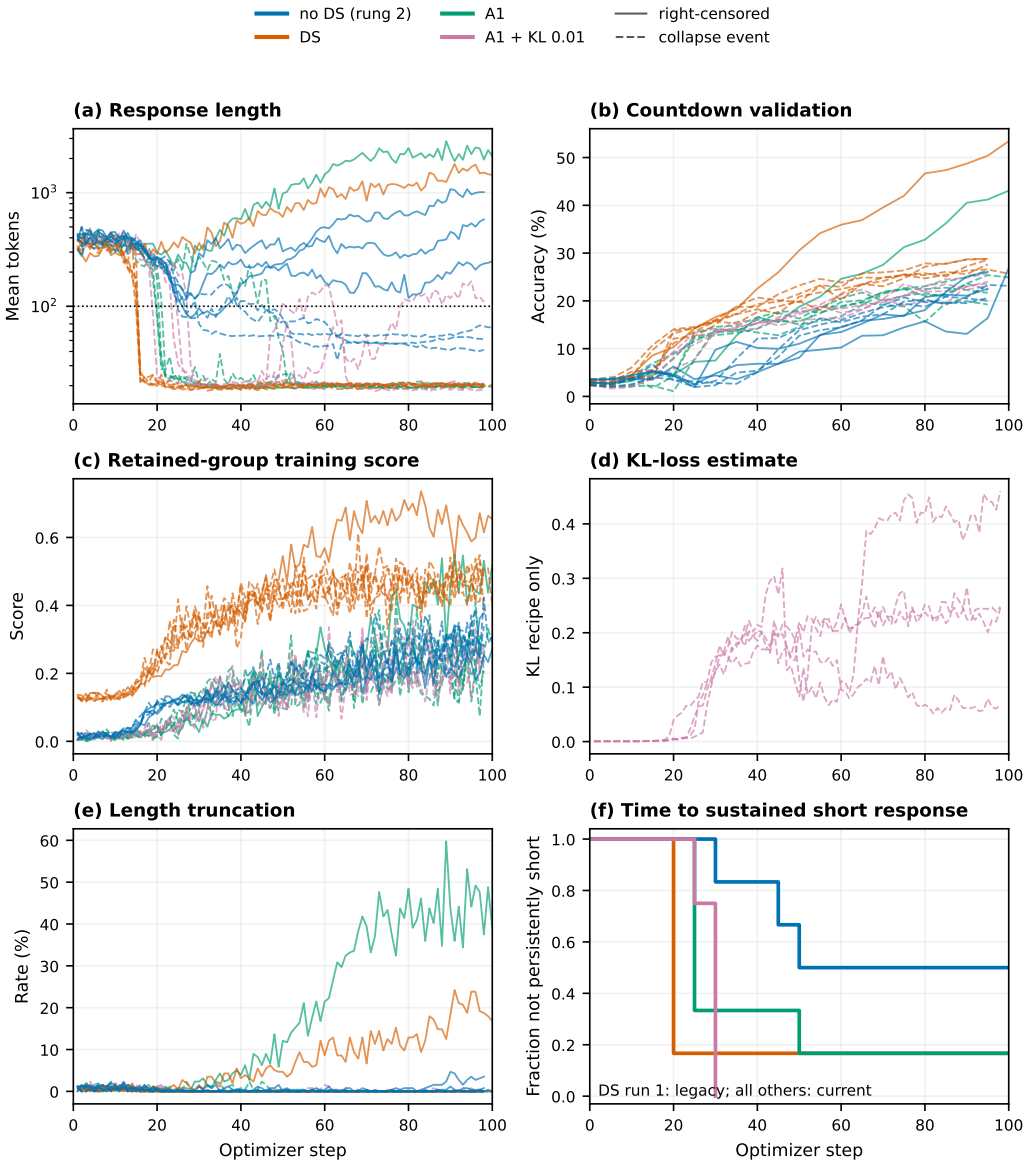


Figure 1. First-100-step joint diagnostics and time to the first sustained short-response episode for all 22 screened runs. Each thin line is one run; color identifies recipe, solid lines are right-censored, and dashed lines have an event. Panel letters identify response length, Countdown validation accuracy, training score, KL-loss estimate, truncation, and the descriptive Kaplan–Meier curve. The response-length panel uses a logarithmic scale and shows the 100-token threshold. Dynamic-sampling training score starts higher because it is computed only over retained groups conditioned on within-group accuracy variation, whereas no-DS includes all generated groups; scores are therefore not compared across recipes. The sole legacy-environment run is the DS run displayed as 1. No confidence bands are shown because each arm has only four to six runs. Token entropy and explicit format-validity rates were not logged and are not inferred.

The joint panels reinforce the endpoint's limited construct validity. Several short-response runs retained 20–29% Countdown validation accuracy, while the healthy A1 and legacy DS seeds reached 42% and 51%, respectively; the healthy no-DS seeds ranged from 19% to 26%. Long-response survivors could have high truncation rates (approximately 43% for A1 seed 1 and 20% for legacy DS seed 1), so length alone does not distinguish coherent reasoning from unfinished generation. Training-score levels are not compared across recipes because their reward definitions differ. The archived histories contain re-

ward/score, approximate KL, and truncation, but not token entropy or an explicit format-validity metric; those missing quantities cannot be reconstructed from scalar logs.

The step-100 snapshot can make collapse look like mere concision because some collapsed runs then match or exceed healthy runs from other recipes in validation accuracy. The longer archived trajectories in Table 5 show why the label matters empirically: within both current-environment matched no-DS and A1, the collapsed member remains near 29–30% at step 520, while a healthy member reaches 63–70%. The comparison remains observational and response length is not itself a capability measure, but the paired recipe-level divergence rules out treating every short-response endpoint as an equally capable concise style.

Table 5. Long-horizon in-domain outcomes for archived healthy and collapsed runs. Endpoint status is fixed from mean response length over steps 90–100; the reported accuracy is the run's sampled Countdown validation accuracy at the listed checkpoint. The DS current run ended at step 245, so step 200 is its last saved checkpoint.

Recipe	Environment	Endpoint status	Step	Validation accuracy (%)
DS	Legacy	Healthy	300	66.0
DS, seed 2	Current	Collapsed	200	34.0
Matched no-DS, run 1	Current	Collapsed	520	30.2
Matched no-DS, seed 2	Current	Healthy	520	63.5
Matched no-DS, seed 3	Current	Healthy	520	64.1
A1, run 1	Current	Healthy	520	70.0
A1, seed 2	Current	Collapsed	520	28.5

To avoid selecting examples by their apparent quality, Appendix B uses a content-blind rule based only on recipe, seed number, endpoint status, and event time. The selected outputs confirm both sides of the construct-validity problem: sustained short-response episodes often yield wrong answer-only equations, while an endpoint-healthy no-DS seed produces an extremely long unfinished response without an answer tag. Recovery in response length also does not guarantee correctness.

5.4. Dynamic Sampling and Training Exposure

The original DS and no-DS runs do not isolate dynamic sampling. The DS run generated approximately 200 prompts per step, retained 32, collected 256 responses, and made one optimizer update. The original no-DS run generated and trained 192 prompts, collected 1,536 responses, and made six updates per step. Thus, at step 300 the two checkpoints represent approximately 300 versus 1,800 optimizer updates. The matched no-DS ablation used 32 generated and trained prompts, 256 responses, and one update per step.

In one matched run, no-DS collapsed to a 54-token mean response and 25% Countdown validation accuracy at step 300, whereas the legacy-environment DS run retained approximately 2,577-token responses and reached 66.0%. That contrast does not support a general DS effect once the screen and replay are considered: both DS and no-DS produced healthy and collapsed outcomes within the current environment. The evidence therefore does not establish that disabling DS is intrinsically superior. Nor does the single replay determine whether the DS contrast is driven by the data seed, rollout seed, their interaction, or a residual difference between the reconstructed and archived configurations.

5.5. Downstream Transfer

Table 6 reports archived selected checkpoints under the primary format-tolerant metric and adds the only collapsed DS checkpoint evaluated on all four full test sets. Step 300 is used for the legacy DS/no-DS comparison because it is the only archived full checkpoint for the healthy DS run; step 250 is used for the GRPO KL sweep; and the displayed

PPO result is its final step-520 checkpoint. These choices were not preregistered. Except for the explicitly labeled collapsed row, the table is a survivorship case study conditional on checkpoint selection, not an estimate of expected performance from starting a recipe. The two strongest selected checkpoints improved GSM8K by 3.1–3.3 percentage points relative to the base model, whereas healthy DS improved it by 0.8 points. Under the standard prompt, the collapsed DS seed-2 checkpoint was only 1.8 points below the base model on GSM8K and lower on the other three benchmarks. Its failure was strongly prompt-conditional: under the Countdown-style think prompt, strict accuracy fell to 9.9% on GSM8K and 14.7% on MATH, compared with 48.4% and 47.4% for the base model. The run exhausted its finite two-epoch data iterator at step 245 after dynamic resampling required up to three generation rounds per update; with a 50-step save cadence, step 200 was the last available checkpoint. The displayed PPO checkpoint was lower than the base model on all four benchmarks. Other PPO configurations at learning rate 3×10^{-6} degraded substantially further. These comparisons describe complete training recipes, not controlled estimates of PPO, GRPO, or DAPO as algorithms.

Table 6. In-domain and transfer evaluation of selected checkpoints (survivorship case study; not an expectation over recipe seeds). Countdown is the sampled validation accuracy logged during training; downstream full-test-set accuracy uses the standard prompt and format-tolerant scoring. NR denotes a non-comparable logged reward rather than accuracy. Values are individual checkpoints, not seed averages.

Checkpoint (recipe)	Countdown	GSM8K	MATH	BBH	MMLU-Pro
Base model	3.0	72.3	55.3	35.4	33.1
DS, step 300	66.0	73.0	55.2	35.3	33.1
DS seed 2, step 200 (collapsed)	34.0	70.4	54.5	35.1	32.7
Original no-DS, step 300	75.1	75.6	54.9	34.5	32.9
GRPO, KL 0.04, step 250	NR	75.4	55.4	35.2	33.9
PPO, lr 10^{-6}, step 520	NR	62.7	46.2	24.3	31.4

Appendix Table A3 gives the matched-step GRPO KL sweep. Stronger KL is associated with preservation of the requested reasoning behavior, but with one seed per coefficient it is a prompt-conditioned mechanism diagnostic rather than a reliable threshold estimate.

5.6. Format Compliance versus Problem Solving

The strict GSM8K score of the base model was 33.36%, although its format-tolerant accuracy was 72.25%; 54.44% of base responses omitted the expected marker. Table 7 uses tolerant correctness first and treats marker compliance as a separate transition. For all three selected checkpoints, tolerant fixes and losses were both common and largely offset one another; the much larger strict-score increases were dominated by marker compliance among responses that were already correct under tolerant scoring. On MATH, where the base model rarely omitted the marker, strict and tolerant scores were similar and no comparable gain appeared.

Exploratory paired item-level tests on tolerant GSM8K predictions gave exact McNemar $p = 0.498$, 0.0043, and 0.00091 for DS, original no-DS, and step-250 strong-KL GRPO, respectively. Holm adjustment across these three displayed comparisons gives $p = 0.498$, 0.0086, and 0.0027. The paired asymptotic 95% intervals in Table 7 describe benchmark-item uncertainty for fixed checkpoints. The comparisons and outcomes were selected after examining a broader sweep, so even adjusted item-level tests remain exploratory; they do not incorporate training-seed variance and must not be read as confirmatory method-level significance tests.

Table 7. Format decomposition for selected surviving checkpoints (survivorship case study; not an expectation over recipe seeds). Paired GSM8K transitions are percentages of 1,319 items. Tolerant fixes are incorrect-to-correct transitions; tolerant losses are correct-to-incorrect transitions. Marker wins/losses are restricted to items for which both responses are tolerant-correct and only one satisfies the strict final-answer-marker rule.

Checkpoint	Tolerant fixes	Tolerant losses	Net tolerant	Paired 95% CI	Marker wins	Marker losses
DS, step 300	7.05	6.29	0.76	$[-1.21, 2.73]$	30.86	1.52
Original no-DS, step 300	10.31	6.97	3.34	$[1.10, 5.57]$	31.77	1.74
GRPO, KL 0.04, step 250	7.13	4.02	3.11	$[1.31, 4.90]$	33.06	1.44

5.7. Prompt-Only Format Control

To test how much strict-score movement can be obtained without changing model weights, we evaluated the base model with the standard GSM8K prompt plus a mandatory final-line contract: the response had to end with exactly one line of the form `#### N`, with only the final numeric answer replacing `N`. All other evaluation choices were held fixed: the pinned base-model snapshot, Qwen chat template, 1,319 test items, greedy decoding, 8,192-token cap, and strict/tolerant scorers. This control was specified and run after the earlier results had been examined, so it is a diagnostic rather than a preregistered confirmatory experiment. The intervention results and selected trained-checkpoint references are summarized in Table 8.

Table 8. GSM8K prompt-only format control and fixed-checkpoint references. Marker rate is the percentage of responses containing a recognized explicit final-answer marker. Trained-checkpoint rows use the original standard prompt and are descriptive references, not randomized controls.

Condition	Strict accuracy (%)	Tolerant accuracy (%)	Marker rate (%)	Truncated (%)
Base, standard prompt	33.36	72.25	45.56	0.15
Base, mandatory-format prompt	58.53	67.10	88.93	0.45
DS, step 300, standard prompt	66.11	73.01	89.01	0.45
Original no-DS, step 300, standard prompt	69.90	75.59	91.28	0.83
GRPO, KL 0.04, step 250, standard prompt	69.52	75.36	92.19	0.61

Relative to the base model under the standard prompt, the mandatory-format prompt increased strict accuracy by 25.17 points (paired 95% CI $[21.86, 28.48]$; 124 standard-only versus 456 format-control-only correct items; exact McNemar raw $p = 1.45 \times 10^{-45}$, Holm-adjusted across the two reported outcomes $p = 2.91 \times 10^{-45}$). It simultaneously *decreased* tolerant accuracy by 5.16 points (95% CI $[-7.70, -2.61]$; 182 standard-only versus 114 format-control-only; raw and Holm-adjusted $p = 9.23 \times 10^{-5}$). Thus, a stronger inference instruction reproduced a large strict-score gain but did not improve problem correctness. The tolerant reduction also shows that prompting is not behaviorally neutral; this control should not be interpreted as a causal decomposition of the trained models' outputs or as a substitute for a format-only supervised-tuning experiment.

5.8. Pass@k and Solution Coverage

Sampling-based evaluation can distinguish increased probability on sampled solutions from evidence that additional items become solvable; the empirical curves are plotted in Figure 2. On MATH-500, base, DS step 300, original no-DS step 300, and headline strong-KL GRPO step 250 obtained format-tolerant pass@1 of 54.06, 55.23, 54.07, and 54.40, respectively; their pass@64 values were 91.0, 92.2, 90.8, and 91.6. On GSM8K, pass@1 was 70.40 for the base model, 72.92 for DS step 300, 73.87 for original no-DS step 300, and 73.21 for `GRPO_n2_kl4e-2_step250`; pass@32 was 96.29, 96.82, 96.66, and 97.12. The GRPO pass@k curve uses the same step-250 checkpoint as Table 6; an archived step-520 run gives the same qualitative pattern. Differences at pass@1 were smaller at the largest sampled k, which is consistent with probability redistribution but does not prove equality of latent so-

lution support: finite samples, ceiling saturation, correlated generations, truncation, and scorer error remain possible. The selected PPO step-520 checkpoint had lower empirical coverage: GSM8K pass@32 was 88.48 and MATH pass@64 was 85.6.

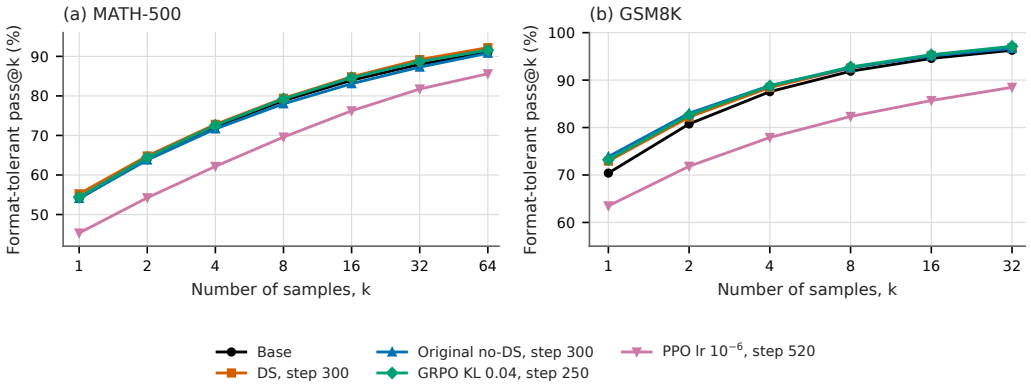


Figure 2. Empirical format-tolerant pass@k for fixed checkpoints on MATH-500 and GSM8K. Curves use up to 64 samples per MATH-500 item and 32 per GSM8K item under the common sampling protocol. The GRPO curve uses the same step-250 checkpoint as the headline transfer table. The convergence of several curves at large k is descriptive and does not establish equal latent solution support.

6. Discussion

The central empirical result is run-dependent instability. Early short-response collapse occurred in 16 of 22 screened runs, often within the first 25 steps. All five current-environment DS screen runs with equal data and rollout seed pairs 2–6 failed identically at the step-20 event window. However, the targeted reconstructed-legacy replay with seed pair (1, 0) remained healthy in that same environment, as did the archived legacy run. The current environment is therefore not a deterministic explanation for DS collapse. The matched no-DS recipe likewise shows healthy and collapsed seeds within the current environment. These data support sensitivity to run-level stochastic or residual configuration differences, but the design does not identify a precise mechanism or population failure rate. The selected long-reasoning checkpoints come from a heterogeneous distribution containing many failures. Reporting only those checkpoints would materially overstate recipe reliability. Tables 6 and 7 estimate performance conditional on selected checkpoints, not expected performance from starting a recipe.

Dynamic sampling cannot be assigned a simple positive or negative effect from these experiments. The original DS/no-DS comparison changes training exposure by a factor of six, while the matched and multi-seed evidence shows collapse with and without DS. Similarly, the group-size experiments increase the number of generated responses from 384 to 768 and 1,536 as G increases, so group size is confounded with sampling budget. These findings are best interpreted as comparisons of complete recipes unless the varied factor and compute exposure are explicitly matched.

The transfer evidence is narrower than the preliminary preprint [13] implied. Under format-tolerant scoring, selected successful runs modestly improve GSM8K but do not consistently improve MATH, BBH, or MMLU-Pro. Much of the large strict-score gain is learned answer formatting. The prompt-only control strengthens that conclusion: an explicit final-line contract raised base-model strict accuracy by 25.2 points while lowering tolerant accuracy by 5.2 points. At the largest sampled k, successful models and the base model have similar empirical coverage, a pattern consistent with RL changing which existing behaviors are sampled first but not proving equal latent capability. This does not

make the gain useless—better pass@1 can be operationally valuable—but it is different from evidence of a broad new reasoning capability.

Several limitations constrain the conclusions. All runs use one 1.5B-parameter model and one narrow arithmetic training domain. Most full-length recipes have one seed. Only one targeted current-environment DS replay used the legacy default seed pair, so it rules out deterministic environment failure but cannot identify the causal random stream, exclude every residual reconstruction difference, or estimate a failure probability. BBH and MMLU-Pro use a custom zero-shot protocol rather than every benchmark's conventional few-shot setting. Checkpoints are evaluated at different steps, and some configurations differ in trainer code, response cap, reward, batch construction, and update count. The format intervention is a prompt-only control and does not isolate what a format-only supervised update would do. Item-level paired tests quantify uncertainty over benchmark items for a fixed checkpoint but not over stochastic training. Full multi-seed runs in one frozen environment, a predetermined checkpoint rule, and compute-matched ablations are required before making causal or algorithm-ranking claims.

These findings suggest four practical safeguards. First, report format-tolerant correctness beside strict answer-marker scores. Second, inspect reward components and group-standardized advantages so that small format credits cannot silently dominate otherwise homogeneous groups. Third, report collapse frequency across seeds before claiming recipe superiority, using a predetermined checkpoint rule where possible. Fourth, complement pass@1 with pass@k and explicit scorer audits to distinguish probability redistribution from evidence of expanded empirical solution coverage.

7. Conclusion

Countdown RL produced large in-domain changes but did not yield uniform general-reasoning improvement in Qwen2.5-1.5B-Instruct. Early short-response collapse was the modal screen outcome. A targeted healthy DS replay in the current environment rules out a deterministic environment explanation for the five matched DS failures, while both DS and no-DS evidence show run-level variation; reward changes, overlong filtering, and KL regularization also failed to prevent collapse uniformly. Among selected non-collapsed checkpoints, the clearest downstream effect was a modest GSM8K pass@1 increase and substantially better answer-format compliance; a prompt-only intervention reproduced a 25.2-point strict-score gain without a tolerant-accuracy gain. MATH, BBH, and MMLU-Pro did not improve consistently. The defensible conclusion is not that one named algorithm is best or that one random stream has been isolated, but that configuration, stochasticity, training exposure, and evaluation protocol jointly determine the apparent outcome.

Author Contributions: Conceptualization, Y.L.; methodology, Y.L.; software, Y.L.; validation, Y.L.; formal analysis, Y.L.; investigation, Y.L.; data curation, Y.L.; writing—original draft preparation, Y.L.; writing—review and editing, Y.L.; visualization, Y.L.; project administration, Y.L. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The public reproducibility artifact—including training and evaluation code, exact configurations, the run registry, compact derived results, vendored training and GSM8K parquet files, immutable revisions or file hashes for every dataset, environment locks, checkpoint hash manifests, and verification scripts—is available in the public GitHub repository. A DOI-bearing archived release will be cited in the final accepted version. Large model checkpoints, raw training logs, and per-item generation archives are available from the corresponding author on rea-

sonable request where redistribution terms permit. Third-party source datasets and the Verl framework are available through their respective citations.

Acknowledgments: During preparation of this manuscript, the author used OpenAI Codex for language editing, manuscript restructuring, and code-assisted consistency checks. The author reviewed and edited all output and takes full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflicts of interest.

Appendix A. Evaluation Prompts, Dataset Identifiers, and Scoring

Chat wrapping and stopping.

Each benchmark prompt below was passed as the content of one user message to the Qwen tokenizer's `apply_chat_template` method with `add_generation_prompt=True`. Greedy evaluation used vLLM with bfloat16 weights, seed 0, temperature 0, and the model's default EOS stopping behavior; generation also stopped at the reported maximum-token limit. Pass@k used the same wrapping with seed 0, temperature 0.6, top-$p = 0.95$, and a 4,096-token cap. The current-environment ablations used Qwen model snapshot `989aa7980e4cf806f80c7fef2b1adb7bc71aa306`; the legacy DS/no-DS runs used a local model copy whose content hash was not recorded.

Checkpoint content identifiers. In `eval/CHECKPOINT_MANIFEST.json`, each aggregate is the SHA-256 of a canonical list containing every relative file path, byte count, and file SHA-256 in the merged Hugging Face directory. The aggregate identifiers are `4adfe72afb42a58da13e5e7adc4c6b5281da07e3ab31a60d6db39796b2160752` for DS step 300, `e1a6abf1c9debe0928f670ed804b4152a506a1c0e79f32fd0c7006b12cfe392d` for original no-DS step 300, `679f16c43306f74cd0c94444c4e6c2f8472acccba062d11f2c469c8c5713ab4b` for strong-KL GRPO step 250, and `d67441ed7d3f8378ebc839c0eb9c613109c335debbbd1e822cbf0851534203bf` for PPO step 520. Each directory contains nine files totaling 3,570,094,814 bytes. The manifest preserves individual file hashes so a public checkpoint deposit can be verified independently.

Exact prompt strings. Braced fields denote item-specific substitutions. The standard GSM8K user content was:

```
{question} Let's think step by step and output the final answer after "####".
```

The prompt-only format control appended the following text to that standard user content:

```
Your response must end with exactly one separate final line of the form
"#### N", where N is replaced by only the final numeric answer. This final
line is mandatory and must be the last line of your response.
```

The standard MATH user content was:

```
{problem}
Please reason step by step, and put your final answer within \boxed{}.
```

The standard BBH user content was:

```
{input}

Let's think step by step, and end your response with
"So the answer is <answer>."
```

The standard MMLU-Pro user content was:

```
The following is a multiple choice question about {category}. Think step by
step and then finish your answer with "the answer is (X)" where X is the
correct letter choice.
```

```
Question: {question}
Options:
{lettered options}
Answer: Let's think step by step.
```

The common think-condition string, including the wording and quotation marks inherited from the training wrapper, was passed as a *single user-message content string*:

```
system: You are a helpful assistant. You first thinks about the reasoning
process in the mind and then provides the user with the answer."
User: {question} Show your work in <think> </think> tags. And return the
final answer in <answer> </answer> tags, for example
<answer> {example} </answer>."
Assistant: Let me solve this step by step.
<think>
```

Dataset identifiers.

GSM8K used the 1,319-item test parquet distributed with the local Verl data preparation. Its upstream revision was not retained, but the exact evaluated file is vendored as `eval/data/gsm8k_test.parquet` with SHA-256 `f284e0ad015991d0059d265051a3c0915c5c1def15a8c01ac9a9142eeb87a180`. Exact comparison of every saved input/reference row with locally cached immutable snapshots recovered the other revisions: MATH used all seven test configurations from `EleutherAI/hendrycks_math` at `21a5633873b6a120296cce3e2df9d5550074f4a3` (5,000/5,000 rows); BBH used all 27 test configurations from `SaylorTwift/bbh` at `b5306be6f827cfafbb545ff5a51f96916029b0fd` (6,511/6,511 rows); and MMLU-Pro used the test split of `TIGER-Lab/MMLU-Pro` at `dd36ce4b34827164989f100331f82c5a29741747` (12,032/12,032 rows). The MATH-500 pass@*k* subset is an exact 500-item subset of that pinned MATH snapshot and is preserved as `eval/data/math500.jsonl`, SHA-256 `200806fb17234213e909649c09a14ce01ef836b32d3bcb1a261fcf96cda4470f`. The machine-readable manifest `eval/DATASET_MANIFEST.json` records canonical input digests, and `eval/verify_dataset_manifest.py` checks the archived files offline and can reconstruct the three revision-pinned datasets.

Scorer precedence. For GSM8K, strict scoring selects the last occurring explicit marker among `####`, `\boxed{}`, and `<answer>`. Tolerant scoring accepts a correct strict marker; otherwise it uses the last numeric string only when the response did not end by length truncation. For MATH, strict scoring symbolically verifies the last explicit `\boxed{}` or `<answer>` content, while tolerant scoring applies the same symbolic verifier to expressions parsed from a completed, non-length-truncated response. For BBH and MMLU-Pro, strict scoring selects the last recognized final-answer marker and tolerant scoring otherwise selects the last parenthesized option letter only for completed, non-length-truncated responses; BBH additionally applies task-specific normalization for Dyck-language and word-sorting answers. The saved per-item files contain the raw response, extracted strict and tolerant predictions, correctness flags, and finish reason so that extraction can be re-audited without regenerating model responses.

Appendix B. Systematically Selected Collapse-Screen Outputs

Selection used metadata only, never response content. Among locally archived screen seeds 3–6, each recipe contributed the lowest-numbered endpoint-collapsed seed at its first sustained short-response event and, when available, the lowest-numbered endpoint-healthy seed at step 100. The recovered A1+KL seed is shown at both its earlier event and the endpoint. All seven selections happen to use the fixed first validation item

(numbers 99, 26, 26; target 100). Exact unabridged responses, source-log paths, and the extraction script are provided in `ablation/representative_outputs.md`, `ablation/representative_outputs.json`, and `ablation/extract_representative_outputs.py`. Representative response excerpts are listed in Table A1.

Table A1. Content-blind examples from the collapse screen. "Endpoint" is the primary steps-90–100 label; the selected step is either the first sustained event or step 100. Character count is reported because the archived printed examples do not retain per-example token counts.

Recipe	Seed	Step	Chars	Score	Observation
Matched no-DS	5	30	132	0.1	Tagged but incorrect expression; endpoint collapsed.
Matched no-DS	3	100	12,252	0.0	Long unfinished search with no answer tag; endpoint healthy.
DS	3	20	33	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1	3	25	33	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1 + KL 0.01	3	30	31	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1 + KL 0.01	4	25	31	0.0	Answer-only but incorrect expression during an early event.
A1 + KL 0.01	4	100	173	0.0	Longer explanation after recovery, but the equation remains incorrect; endpoint healthy.

Appendix C. Complete Collapse Screen and Exploratory Grids

Table A2 reports every run in the fixed 22-run screen rather than only representative outcomes. Event step is the first sustained short-response window; a dash denotes right-censoring at step 100. The length and accuracy columns average available observations over steps 90–100.

Table A2. Complete 22-run collapse screen. The asterisk marks the legacy DS run whose stored random seeds are unknown; "1" is only a display identifier.

Recipe	Run/seed	Environment	Event step	Endpoint	Mean tokens	Validation accuracy (%)
DS	1*	Legacy	–	Healthy	1569.4	50.8
DS	2	Current	20	Collapsed	20.4	26.2
DS	3	Current	20	Collapsed	20.6	27.9
DS	4	Current	20	Collapsed	20.5	26.6
DS	5	Current	20	Collapsed	20.3	28.7
DS	6	Current	20	Collapsed	19.5	28.8
Matched no-DS	1	Current	45	Collapsed	61.9	22.4
Matched no-DS	2	Current	–	Healthy	229.6	18.8
Matched no-DS	3	Current	–	Healthy	1007.1	21.8
Matched no-DS	4	Current	–	Healthy	497.6	25.6
Matched no-DS	5	Current	30	Collapsed	41.7	20.3
Matched no-DS	6	Current	50	Collapsed	52.7	20.1
A1	1	Current	–	Healthy	2223.7	41.6
A1	2	Current	25	Collapsed	19.7	24.6
A1	3	Current	25	Collapsed	19.7	22.0
A1	4	Current	50	Collapsed	20.7	22.9
A1	5	Current	25	Collapsed	19.6	19.6
A1	6	Current	25	Collapsed	19.7	25.7
A1 + KL 0.01	3	Current	30	Collapsed	20.9	23.4
A1 + KL 0.01	4	Current	25	Healthy	132.2	25.3
A1 + KL 0.01	5	Current	30	Collapsed	18.5	22.8
A1 + KL 0.01	6	Current	30	Collapsed	20.7	23.5

Table A3. Archived exploratory PPO and GRPO grids. Panel A reports the mean standard-prompt, format-tolerant accuracy across GSM8K, MATH, BBH, and MMLU-Pro. Panel B reports strict accuracy and the range of median think-prompt response lengths across the four benchmarks. Each row is one training run; these incomplete grids do not isolate components or estimate seed variance.

Panel A: PPO reference recipes

Actor LR	Entropy	KL-loss setting	Step	Four-task mean (%)
10^{-6}	0	Off	520	41.2
3×10^{-6}	0	Off	520	19.5
3×10^{-6}	0.001	Off	520	5.9
3×10^{-6}	0.001	Enabled, coefficient 0	520	18.6
3×10^{-6}	0.001	Enabled, coefficient 0.001	520	23.3

Panel B: GRPO KL sweep ($G = 2$, step 250)

KL coefficient	GSM8K std.	GSM8K think	MATH std.	MATH think	Think median chars
Off	44.0	11.1	52.9	15.8	20–22
0.001	49.6	9.8	54.6	14.6	20–22
0.005	47.2	9.6	54.3	15.9	20–22
0.01	56.5	72.9	56.0	52.5	741–1895
0.04	69.5	57.7	55.3	51.7	729–1671

References

1. OpenAI. Learning to Reason with LLMs. Available online: https://openai.com/index/learning-to-reason-with-llms/ (accessed on 12 October 2026).
2. Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv* **2025**, arXiv:2501.12948.
3. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.
4. Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.K.; Wu, Y.; et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv* **2024**, arXiv:2402.03300.
5. Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, L.; et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv* **2025**, arXiv:2503.14476.
6. Hu, J.; Liu, J.K.; Xu, H.; Shen, W. REINFORCE++: An Efficient RLHF Algorithm with Robustness to Both Prompt and Reward Models. *arXiv* **2025**, arXiv:2501.03262.
7. Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W.S.; Lin, M. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv* **2025**, arXiv:2503.20783.
8. Zheng, H.; Zhou, Y.; Bartoldson, B.R.; Kailkhura, B.; Lai, F.; Zhao, J.; Chen, B. Act Only When It Pays: Efficient Reinforcement Learning for LLM Reasoning via Selective Rollouts. *arXiv* **2025**, arXiv:2506.02177.
9. Cui, G.; Zhang, Y.; Chen, J.; Yuan, L.; Wang, Z.; Zuo, Y.; Li, H.; Fan, Y.; Chen, H.; Chen, W.; et al. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. *arXiv* **2025**, arXiv:2505.22617.
10. Shao, R.; Li, S.S.; Xin, R.; Geng, S.; Wang, Y.; Oh, S.; Du, S.S.; Lambert, N.; Min, S.; Krishna, R.; et al. Spurious Rewards: Rethinking Training Signals in RLVR. *arXiv* **2025**, arXiv:2506.10947.
11. Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Song, S.; Huang, G. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? *Adv. Neural Inf. Process. Syst.* **2025**; arXiv:2504.13837.
12. Huan, M.; Li, Y.; Zheng, T.; Xu, X.; Kim, S.; Du, M.; Poovendran, R.; Neubig, G.; Yue, X. Does Math Reasoning Improve General LLM Capabilities? Understanding Transferability of LLM Reasoning. *arXiv* **2025**, arXiv:2507.00432.
13. Lian, Y. Comparative Analysis and Parametric Tuning of PPO, GRPO, and DAPO for LLM Reasoning Enhancement. *arXiv* **2025**, arXiv:2512.07611.
14. Ilyas, A.; Engstrom, L.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; Madry, A. Are Deep Policy Gradient Algorithms Truly Policy Gradient Algorithms? *arXiv* **2018**, arXiv:1811.02553.
15. Wang, Y.; He, H.; Tan, X. Truly Proximal Policy Optimization. *Proc. Mach. Learn. Res.* **2020**, *124*, 113–122.

16. Qwen Team. Qwen2.5: A Party of Foundation Models. Available online: https://qwenlm.github.io/blog/qwen2.5/ (accessed on 12 October 2026).
17. Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. Training Verifiers to Solve Math Word Problems. *arXiv* **2021**, arXiv:2110.14168.
18. Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. *arXiv* **2021**, arXiv:2103.03874.
19. Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H.W.; Chowdhery, A.; Le, Q.V.; Chi, E.H.; Zhou, D.; et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv* **2022**, arXiv:2210.09261.
20. Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv* **2024**, arXiv:2406.01574.
21. Pan, J. Countdown-Tasks-3to4 Dataset. Available online: https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4 (accessed on 21 February 2025).
22. Pan, J.; Zhang, J.; Wang, X.; Yuan, L.; Peng, H.; Suhr, A. TinyZero: Minimal Reproduction of DeepSeek R1-Zero. Available online: https://github.com/Jiayi-Pan/TinyZero (accessed on 12 October 2026).
23. Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; Wu, C. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv* **2024**, arXiv:2409.19256.
24. Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.I.; Moritz, P. Trust Region Policy Optimization. *Proc. Mach. Learn. Res.* **2015**, *37*, 1889–1897.
25. Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.I.; Abbeel, P. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv* **2015**, arXiv:1506.02438.
26. Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; Meger, D. Deep Reinforcement Learning That Matters. *Proc. AAAI Conf. Artif. Intell.* **2018**, *32*, 3207–3214. https://doi.org/10.1609/aaai.v32i1.11694.
27. Engstrom, L.; Ilyas, A.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; Madry, A. Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. In *Proceedings of the International Conference on Learning Representations*; 2020.
28. Agarwal, R.; Schwarzer, M.; Castro, P.S.; Courville, A.C.; Bellemare, M.G. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 29304–29320.